Дорожная карта развития BIS ERP

Дата актуализации: октябрь 2026 года

1. Текущее состояние проекта на июне – октябре 2026 года

1.1 Архитектура

- **Технологический стек:** WPF/.NET 8, PostgreSQL (Npgsql + EF Core), PDF — QuestPDF, Excel — ClosedXML, DBF/FRX — DotNetDBF + собственные парсеры.
- **Мета-архитектура:** справочники, документы, отчёты, модули, поля и правила проводок описываются в базе данных (таблицы `MetadataObjects`, `MetadataFields`, `MetadataCalculations`, `MetadataPostingRules`, `MetadataModules`, `MetadataModuleItems`) и отображаются в рабочем режиме и конфигураторе.
- **Два режима:** рабочее окно (`MainWorkWindow`) и конфигуратор (`ConfiguratorWindow`).
- **Механизм патчей:** зашифрованные `.bispatch` (AES-GCM) с `schema.sql`, `metadata.json`, `reports.json`, `modules.json`, `data.json`.

1.2 Модули

Система поддерживает 8 модулей. По умолчанию активен только модуль **Финансы**. Остальные включаются администратором через `Инфобазы → Модули`.

Код	Название	CloseOrder	Статус
`Finance`	Финансы	900	✅ Активен по умолчанию
`Balance`	Баланс (итоговое закрытие)	10000	⚙️ Служебный этап
`FixedAssets`	Основные средства	200	🧪 Метаданные + тесты
`Inventory`	Материалы	300	🧪 Метаданные + тесты
`Payroll`	Зарплата	400	⏳ Не начат
`Sales`	Сбыт	500	⏳ Не начат
`RawMaterials`	Сырьё	600	⏳ Не начат
`CostAccounting`	Себестоимость	700	⏳ Не начат

1.3 Что уже реализовано

Модуль «Финансы» (Production Ready):

- План счетов и аналитика по счетам.
- Единый журнал проводок `doc_postings`.
- ОСВ, главная книга, баланс, финансовые результаты с формульными строками.
- Кассовые ордера (ПКО/РКО в одном документе `doc_cash_orders`).
- Платёжные поручения.
- Счёт-фактуры (выписка/регистрация) с ЭСФ XML, НДС-журнал, экспорт STI-062_7.
- Авансовый отчет, платёжная ведомость, расчёт курсовой разницы.
- Закрытие периода: сбор → закрытие модулей → итоговый баланс.
- Блокировка документов в закрытом периоде.
- 10 smoke-тестовых сценариев, все `verify` проходят.

Подготовленные (на уровне метаданных и тестов):

- **Основные средства:** каталог «Основные средства» (~39 полей), 8 вспомогательных справочников, 17 документов, 10+ отчётов, модель `FixedAssetPeriodBalance`, автоматическое начисление амортизации во время `CollectAsync`, smoke-тест `fixed-assets-smoke`.
- **Учёт материальных ценностей:** каталог «Справочник материалов», 5 документов, отчёт «Ведомость наличия материалов».

Что есть в модуле «Отчёты и печатные формы»:

- Модель `Report` с ~30 настройками (разметка страницы, шрифты, header/footer, цвета, группировки, фильтры).
- `ReportDesignerWindow` — визуальный конструктор (вкладки: общие параметры, поля, FRX-поля, нативный макет, оформление, фильтры).
- FRX/FRT-парсер, `FoxExpressionParser`, `FrxToReportConverter`.
- `PrintFormService` — генерация печатных форм и PDF.
- `ReportPreviewWindow` (legacy-табличный preview, скрыт в рабочем режиме).
- `PdfPreviewWindow` — внешний PDF-просмотрщик.
- 22 стандартных отчёта (finance., *fixed-assets.*), из которых native-отчёты помечены `IsActive = false` до готовности постраничного вывода.

1.4. Итоги сентября 2026 года

В сентябре 2026 года **не удалось реализовать большинство запланированного**. Основная часть месяца ушла на проработку фундаментальных вопросов на этапе

подготовки, что выявило ряд проблем и потребовало пересмотра принятых технических решений:

Направление	С чем столкнулись	Принятое решени
Модели данных	Уточнялись модели данных: состав поля, типы, связи и принадлежность сущностей модулям (ОС, ТМЦ, отчётность), согласование схем с метаданными и правилами проводок заняло больше времени, чем планировалось, и блокировало начало работ высшего уровня.	Проведён пересм и доработка моде данных; выход на бизнес-логику отложен до стабилизации схе
Документирование проекта	Выявились расхождения документации с фактическим состоянием кода. Актуализация архитектурных документов (`ARCHITECTURE` , `PROJECT_SCHEMA` , `ROADMAP` и профильных) оказалась объёмной задачей, не заложенной в план.	Документация приведена в соответствие с код зафиксировано правило обновлят документы одновременно с изменением функциональност
Отчётность FRX	Существующий подход на базе FRX-конвертера **не подошёл** под требования системы: ограничения формата макета, вложенных выражений и постраничного вывода не позволяют получить нужное качество печатных форм.	Принято решение **разрабо собственный конструктор отчётов** — нативн конвейер `DataSou` → `Template` → `Renderer` → `Output` (раздел 3 Проектирование начато в сентябре

Влияние на график: реализация рабочего UI модулей «Основные средства» и «Учёт материальных ценностей» перенесена на **октябрь – ноябрь 2026 года**, весь последующий план работ сдвинут на один месяц. Разделы 2–5 настоящей дорожной карты актуализированы с учётом этого сдвига.

---

2. Дорожная карта по модулям

> **Принцип построения:** каждый модуль проходит пять стадий:
>
> 1. **Метаданные** — каталоги, документы, поля, проводки, отчёты в базе.
> 2. **Бизнес-логика** — проведение документов, расчёты, движения, автоматические операции.
> 3. **UI** — рабочее окно модуля, справочники, формы документов, диалоги.
> 4. **Период закрытия** — сбор оборотов, снимки, проверка сбалансированности.
> 5. **Тесты** — smoke-тесты и строгие числовые проверки.

2.1. Модуль «Финансы» — текущие задачи (октябрь)

Статус: Production Ready, но есть технический долг.

Элемент	Статус	Что делаем
Строгие числовые тесты	❌	Добавить для кассы, ПП, авансового отчёта, платёжной ведомости, курсовой разницы
Загрузка курсов НБКР	❌	Довести команду администратора
Баланс/ФР	⚠️	Базовые строки есть; нужен полный перенос Fox-формул
Эталонные данные	❌	Сбор за ≥3 периода: ОСВ, главная книга, журнал НДС, СФ

2.2. Модуль «Основные средства» — реализация в октябре – ноябре 2026 г.

Статус сейчас: Метаданные + бизнес-логика в `AccountingPeriodService` + smoke-тест. Документы открываются через универсальный `DynamicDocumentItemDialog.BuildFormAsync`; для «Учета движения ОС»

уже есть кастомные построители (движение, ввод в эксплуатацию, передача под отчет). **Нет** рабочего окна модуля (`FixedAssetsWorkView`) и навигации.

Этап 1. Метаданные (✅ уже готово)

- Каталог «Основные средства» (39 полей) ✅
- 8 вспомогательных справочников ✅
- 17 документов с полями и правилами проводок ✅
- ReportSource «Снимки остатков ОС» ✅
- 10 отчётов в `BuildStandardReportDefinitions` ✅
- Константа `FixedAssetDocumentNames` ✅

Этап 2. Бизнес-логика (✅ частично готово)

- `EnsureFixedAssetPeriodClosingAsync` — генерация авто-амортизации ✅
- `SaveFixedAssetPeriodBalancesAsync` — расчёт всех показателей ОС ✅
- `FixedAssetPeriodMovement` — движения по стоимости/амортизации ✅
- Проведение документов через `MetadataService.PostDocumentAsync` ✅

Этап 3. UI (🟡 октябрь–ноябрь — основной объём работы)

Ключевое решение (переработано): отдельные XAML-диалоги под каждый документ ОС **не создаём**. Все документы (включая «Передача под отчет») добавляются как **метаданные** и открываются через единый универсальный диалог `DynamicDocumentItemDialog` , а форма строится автоматически в `BuildFormAsync` . Новый документ = (1) `MetadataObject` + поля + правила проводок, (2) регистрация в `ModuleMetadataService.FixedAssetDocumentNames` , (3) при необходимости — точечная доработка `BuildFormAsync` (для документов с табличной частью).

Единый конвейер добавления документа ОС (на примере «Передача под отчет»):

```
ModuleMetadataService.FixedAssetDocumentNames
        │  (имя документа)
        ▼
MetadataObjects: объект типа Document (таблица doc_…)
        │  + MetadataFields: Номер/Дата/Примечание, реквизиты,
        │    ReferenceCatalog → «Основные средства», «Сотрудники (МОЛ)»,
«Участки»…
        │  + MetadataPostingRules: правила проводок
        ▼
SynchronizeDefaultAssignmentsAsync / AssignMissingByNameAsync → модуль
«FixedAssets»
        ▼
```

```
DynamicDocumentWorkView (журнал документов, кнопки Добавить/Провести)
        ▼
DynamicDocumentItemDialog.BuildFormAsync ──► форма документа
   ├─ «Учет движения ОС» → кастомные построители (см. ниже)
   └─ остальные 16 документов → универсальный цикл по _metadata.Fields:
        CreateControlAsync → ReferencePickerControl / ComboBox / Дата /
Сумма / Скрытые
        (номер подставляется автоматически, is_posted — скрытое поле)
        ▼
OnSaveClick → ItemData → CreateDynamicRecordAsync / UpdateDynamicRecordAsync
        ▼
Проведение → MetadataService.PostDocumentAsync →
ProcessFixedAsset*DocumentAsync
```

№	Компонент	Описание
1	`FixedAssetsWorkView.xaml`	Рабочее окно модуля: журнал (
2	`FixedAssetCatalogView.xaml`	Браузер справочника «Основн
3	`FixedAssetDetailsDialog.xaml`	Уже существует — основная ка
4	`DynamicDocumentItemDialog.BuildFormAsync` — документы через метаданные	Все 17 документов ОС рендер метаданные + регистрация в `F`
5	Кастомные построители «Учет движения ОС» в `BuildFormAsync`	`BuildFixedAssetMovementForm` «Передача под отчет», код 13), части `FixedAssetLineFieldAl`
6	Навигация в `MainWorkWindow`	Пункт «Основные средства» в

Порядок реализации UI в октябре – ноябре:

1. **Неделя 1:** `FixedAssetCatalogView` + `FixedAssetDetailsDialog` — просмотр/создание/редактирование карточек ОС, поддержка групп/подгрупп/видов.
2. **Неделя 2:** Документы через `DynamicDocumentItemDialog.BuildFormAsync`. Для каждого из 16 «обычных» документов — проверить метаданные (поля, порядок, ссылки) и провести ручной прогон формы; для «Учета движения ОС» — кастомные построители (движение, ввод в эксплуатацию, **передача под отчет**).
3. **Неделя 3:** `FixedAssetsWorkView` — главное окно модуля: вкладки «Справочник ОС», «Документы», «Отчёты».
4. **Неделя 4:** Подключение модуля в навигацию `MainWorkWindow`, включение модуля по умолчанию в тестовых инфобазах, интеграция с периодом закрытия.

Этап 4. Период закрытия (✅ готово)

- `FixedAssets` имеет `CloseOrder = 200`, `ParticipatesInPeriodClose = true`.
- В `CollectAsync` вызываются `EnsureFixedAssetPeriodClosingAsync` и `SaveFixedAsse tPeriodBalancesAsync`.
- Модуль закрывается перед `Finance` (CloseOrder 200 < 900).

Этап 5. Тесты (✅ smoke-тест, 🟡 строгие числа)

- `fixed-assets-smoke verify/run/cleanup` — полный цикл: покупка → ввод → амортизация → переоценка → передача → выбытие → закрытие.
- **Нужно в октябре – ноябре:** добавить строгие числовые проверки (аналогично `finance-registers`) для сверки начисленной амортизации, остаточной стоимости и проводок.

Документы, которые будут доступны пользователю в октябре – ноябре:

Все формы строятся автоматически в `DynamicDocumentItemDialog.BuildFormAsync`:

- **Универсальный цикл** — документ без табличной части рендерится по полям из метаданных (требуется только качественно настроенные поля/порядок/ссылки).
- **«Учет движения ОС»** — общий документ-конверт: вид движения выбирается из справочника «Ввод нового документа ОС», а `BuildFormAsync` по подстроке вида открывает нужный построитель формы.

№	Наименование	Таблица	Кратко
1	Покупка ОС	`doc_fixed_asset_purchase`	Приход ОС по договору/счёту
2	Ввод ОС в эксплуатацию	`doc_fixed_asset_commissioning`	Ввод в эксплуатацию
3	Вывод из эксплуатации	`doc_fixed_asset_decommissioning`	Снятие с учёта
4	Переоценка ОС	`doc_fixed_asset_revaluation`	Переоценка стоимости
5	Реализация ОС	`doc_fixed_asset_sales`	Полное выбыти
6	Частичная реализация ОС	`doc_fixed_asset_partial_sales`	Частичное выбытие
7	Начисление амортизации	`doc_fixed_asset_depreciation`	Ручное/авто начисление
8	Списание амортизации	`doc_fixed_asset_depreciation_writeoff`	Списание амортизации

№	Наименование	Таблица	Кратко
9	Консервация ОС	`doc_fixed_asset_conservation`	Приостановка амортизации
10	Расконсервация ОС	`doc_fixed_asset_reopening`	Возобновление амортизации
11	Передача ОС в подотчёт	`doc_fixed_asset_to_employee`	Передача МОЛ
12	Смена затратного счета	`doc_fixed_asset_account_change`	Изменение счета расходов
13	Смена группы ОС	`doc_fixed_asset_group_change`	Смена классификатора ОС
14	Укомплектация ОС	`doc_fixed_asset_assembly`	Сборка из деталей
15	Разукомплектация ОС	`doc_fixed_asset_disassembly`	Разборка на детали
16	Приход из производства ОС	`doc_fixed_asset_from_production`	Поступление из Производства
17	Учёт движения ОС	`doc_fixed_asset_movement`	Специальный документ учёта движения

> Обозначения форм: **Универсальный цикл** — общая ветка `BuildFormAsync` (поля из метаданных, авто-подстановка номера, скрытое `is_posted`); **+ строка ОС** — табличная часть строк основного средства (`FixedAssetLineFieldAliases` / `FixedAssetCommissioningLineFieldAliases`), которую также дорабатываем прямо внутри `BuildFormAsync`, без отдельных XAML.

2.3. Модуль «Учёт материальных ценностей» — реализация в октябре – ноябре 2026 г.

Статус сейчас: Есть модель `Material`, каталог, 5 документов в метаданных, 1 отчёт. **Нет** бизнес-логики проводок, UI и периода закрытия.

Этап 1. Метаданные (✅ частично)

- Каталог «Справочник материалов» ✅ (модель `Material`)
- 5 документов в `DocumentationMetadataSeedService` ✅
- Отчёт «Ведомость наличия материалов» ✅

Этап 2. Бизнес-логика (🟡 октябрь–ноябрь)

Нужно реализовать:

- Проводки для прихода ТМЦ (Дт счёт склада / Кт корр. счёт).
- Проводки для расхода ТМЦ в производство (Дт счёт расходов / Кт счёт склада).
- Проводки для перемещения (Дт/Кт счета складов).
- Проводки для передачи в подотчёт (Дт расход / Кт счёт ТМЦ).
- Проводки для инвентаризации (переоценка остатков).
- Автоматическая генерация проводок через `MetadataService.PostDocumentAsync`.

Этап 3. UI (🟡 октябрь–ноябрь)

- `InventoryWorkView.xaml` — журнал ТМЦ и документы.
- `InventoryDocumentDialog.xaml` — форма всех 5 документов.
- Подключение в навигацию `MainWorkWindow`.

Этап 4. Период закрытия (🟡 октябрь–ноябрь)

- **Нужно создать:** `InventoryPeriodBalance` модель и таблицу.
- Интеграция в `CollectAsync` (аналогично `SaveFixedAssetPeriodBalancesAsync`).
- `Inventory` имеет `CloseOrder = 300`.

Этап 5. Тесты (🟡 октябрь–ноябрь)

- `inventory-smoke` — сценарий: приход → перемещение → расход → инвентаризация → сверка остатков.

Документы модуля:

№	Наименование	Кратко
1	Приход из производства ТМЦ	Приход ТМЦ на склад
2	Расход в производство	Расход ТМЦ на производство
3	Внутреннее перемещение ТМЦ	Перемещение между складами
4	Передача ТМЦ в подотчёт	Выдача сотруднику
5	Инвентаризация ТМЦ	Перепроверка остатков

2.4. Модуль «Зарплата» — планы на ноябрь 2026 – февраль 2027

Этап	Что	Примерный срок
1	Метаданные: справочники (сотрудники, должности, тарифы), документы (начисление, выплата, удержания)	Ноябрь
2	Бизнес-логика: расчёт ЗП, НДФЛ, страховых взносов	Декабрь
3	UI: `PayrollWorkView`, диалоги документов	Январь
4	Период закрытия + тесты	Февраль

2.5. Модуль «Сбыт» — планы на декабрь 2026 – март 2027

Этап	Что	Примерный срок
1	Метаданные: справочники (номенклатура, цены), документы (реализация, возврат)	Декабрь
2	Бизнес-логика: ценообразование, скидки, НДС	Январь
3	UI: `SalesWorkView`	Февраль
4	Период закрытия + тесты	Март

2.6. Модули «Сырьё» и «Себестоимость» — планы на апрель – июнь 2027 г.

Этап	Что	Примерный срок
1	Метаданные + бизнес-логика себестоимости продукции	Апрель
2	UI и период закрытия	Май
3	Интеграция с модулями «Сбыт» и «Производство»	Июнь

---

3. Единый генератор отчётности

3.1. Текущее состояние

Сейчас в системе **нет единого генератора отчётности**. Отчёты формируются через разрозненные пути:

Путь	Компонент	Формат вывода	Статус
FRX → Report	`FrxToReportConverter`	FRX-макет → native	⛔ Не подошёл под требования — заменяется собственным конструктором
Native table	`ReportPreviewWindow`	Таблица внутри WPF	❌ Скрыт
Native постраничный	(не реализовано)	Native pagination	⏳ В планах
PDF печатные формы	`PrintFormService` + QuestPDF	PDF	✅ Рабочий
Excel	`ClosedXML`	XLSX	✅ Рабочий (STI-062_7)

Проблемы текущего подхода:

1. `ReportPreviewWindow` — legacy-просмотр, скрыт, не используется.
2. Native-отчёты создаются в метаданных как неактивные (`IsActive = false`) до реализации постраничного вывода.
3. FRX-варианты актов сверки хранятся отдельными печатными формами, но не должны быть отдельными пунктами меню.

4. Каждый тип отчёта (TrialBalance, GeneralLedger, BankStatement, FixedAssetStatement...) имеет свой код рендеринга в `ReportService`.
5. **Формат FRX не подошёл** под требования системы: ограничения макета, вложенных выражений и постраничного вывода не дают нужного качества печатных форм. В сентябре 2026 принято решение разрабатывать **собственный конструктор отчётов**; FRX-конвертер сохранён лишь как средство первичного переноса форм.

Таблицы и модели отчётности:

Таблица	Модель	Назначение
`Reports`	`Report`	Справочник отчётов
`ReportFields`	`ReportField`	Поля отчёта
`ReportFilters`	`ReportFilter`	Фильтры
`ReportGroups`	`ReportGroup`	Группировки
`ReportElementMappings`	`ReportElementMapping`	Элементы FRX-макета
`ReportDataSets`	`ReportDataSet` / `ReportDataSetField`	Источники данных
`ReportHeadersFooters`	`ReportHeaderFooter`	Шапки/подвалы

Стандартные отчёты (22 шт.):

Финансы (13): `postings-journal`, `trial-balance`, `general-ledger`, `bank-statement`, `payment-order-registry`, `reconciliation-act`, `sales-invoice-journal`, `purchase-invoice-journal`, `cash-order-journal`, `cash-order-turnover`, `advance-report`, `payment-order`, `exchange-rate-difference`.

Основные средства (10): `list`, `balances`, `by-account`, `depreciation`, `card`, `journal`, `responsible-person`, `directory`, `inventory-transfer`, `movement`.

3.2. План создания единого генератора отчётности

Этап 1. Проектирование архитектуры (октябрь 2026; начато в сентябре)

Цель: создать единый конвейер `DataSource → Template → Renderer → Output`, заменяющий все текущие разрозненные пути.

Шаг	Что делаем
1.1	Спроектировать интерфейсы `IReportDataSource`, `IReportTemplate`, `IReportRenderer`, `IReportO`
1.2	Определить перечень поддерживаемых форматов: Native table, Native paginated, F Excel, HTML
1.3	Спроектировать DSL шаблонов (JSON-описание макета: секции, строки, колонки,
1.4	Согласовать, как FRX-варианты будут интегрированы (как `IsPrintForm` с `ParentF`

Этап 2. Ядро генератора (ноябрь – декабрь 2026)

Шаг	Что делаем	Срок
2.1	`ReportDataSourceService` — загрузка данных из таблиц, представлений и `doc_postings` по метаданным источника	2 недели
2.2	`ReportTemplateParser` — парсер JSON-шаблонов, построение `ReportDocument` (страницы → секции → строки → ячейки)	2 недели
2.3	`NativeReportRenderer` — движок native-отчётов с постраничным выводом (замена `ReportPreviewWindow`)	3 недели
2.4	`PdfReportWriter` (на базе QuestPDF) — вывод в PDF с разрывами страниц, шапкой/подвалом, нумерацией	2 недели
2.5	`ExcelReportWriter` (на базе ClosedXML) — экспорт в XLSX	1 неделя
2.6	`HtmlReportWriter` — вывод в HTML для просмотра в браузере	1 неделя

Этап 3. Интеграция и миграция (декабрь 2026 – январь 2027)

Шаг	Что делаем	Срок
3.1	Подключить `IReportDataSource` к существующим `ReportDataSet`	1 неделя
3.2	Мигрировать `AccountingReportsView` на новый движок (ТБ, ГК, банковская выписка)	2 недели

Шаг	Что делаем	Срок
3.3	Мигрировать отчёты Fixed Assets (Ведомость, ОСВ, Журнал и др.)	2 недели
3.4	Встроить FRX-варианты как шаблоны внутри `Report`, а не как отдельные печатные формы	2 недели
3.5	Активировать native-отчёты (перевести из `IsActive = false` в `true`)	1 неделя
3.6	Убрать `ReportPreviewWindow` из пути использования	1 неделя

Этап 4. ReportDesigner на новом движке (январь – февраль 2027)

Шаг	Что делаем	Срок
4.1	Добавить вкладку «Native layout» в `ReportDesignerWindow` с WYSIWYG-редактором	2 недели
4.2	Реализовать drag&drop полей, настройку колонок, группировок, фильтров	3 недели
4.3	Превью native-отчёта в реальном времени внутри конструктора	2 недели
4.4	Экспорт шаблона в `.bisreport` (часть патча)	1 неделя

Этап 5. Тесты и документация (февраль 2027)

Шаг	Что делаем	Срок
5.1	Smoke-тесты всех 22 стандартных отчётов на новом движке	2 недели
5.2	Сравнение вывода нового движка с Fox-эталонами	1 неделя
5.3	Документация пользователя и разработчика	1 неделя

3.3. Архитектура единого генератора (проект)

```
┌──────────────────────────────────────────────────────┐
│                  UnifiedReportEngine                 │
├──────────────────────────────────────────────────────┤
│  IReportDataSource  →  ReportDataSourceService       │
│  IReportTemplate    →  ReportTemplateParser (JSON DSL)│
│  IReportRenderer    →  NativeReportRenderer          │
│  IReportOutput      →  PdfReportWriter / Excel / HTML│
├──────────────────────────────────────────────────────┤
│          UnifiedReportEngine.GenerateAsync           │
│                  (reportId, parameters)              │
│  1. загружает Report + ReportFields + Filters        │
│  2. тянет данные через ReportDataSourceService       │
│  3. парсит шаблон в ReportDocument                   │
│  4. рендерит страницы, применяет пагинацию           │
│  5. пишет вывод (PDF / Excel / HTML)                 │
└──────────────────────────────────────────────────────┘
```

Ключевые решения:

- **Один вход:** `GenerateAsync(reportId, parameters)` для всех типов отчётов.
- **Плагинная архитектура:** новые типы рендеринга и вывода добавляются через `IReportRenderer` и `IReportOutputWriter`.
- **FRX-совместимость:** существующие FRX-варианты конвертируются в JSON-шаблоны и становятся шаблонами внутри `Report`.
- **Постраничный вывод:** движок делит контент на страницы с учётом высоты шапки/подвала и высоты строк.
- **Выражения:** переиспользуется `FoxExpressionParser` для вычисления выражений в ячейках.

---

4. Контрольные точки (KPI)

Период	Что должно быть готово	Как проверяется
Сентябрь 2026 ✅ (итог)	Устранены блокеры: пересмотрены и доработаны модели данных, актуализирована документация проекта, пересмотрен подход к	Актуализированные `ROADMAP.md` и профильные документы

Период	Что должно быть готово	Как проверяется
	отчётности (FRX отклонён, принят собственный конструктор отчётов)	
Октябрь – ноябрь 2026	UI модуля ОС (каталог, документы, работа); UI модуля ТМЦ (каталог, документы); smoke-тесты	`fixed-assets-smoke verify` → 0 ошибок; ручной запуск UI
Ноябрь – декабрь 2026	Архитектура единого генератора, ядро движка	Unit-тесты `UnifiedReportEngine`
Декабрь 2026 – январь 2027	Миграция финансовых и ОС-отчётов на новый движок	`finance-reports verify` + ручные проверки
Январь – февраль 2027	ReportDesigner с native layout	Визуальная проверка и экспорт
Февраль 2027	Все 22 стандартных отчёта на новом движке	Smoke-тесты отчётов

---

5. Резюме

Сентябрь 2026 года был посвящён устранению фундаментальных блокеров: пересмотру **моделей данных**, актуализации **документации проекта** и пересмотру подхода к отчётности — формат **FRX не подошёл** под требования системы, поэтому принято решение разрабатывать **собственный конструктор отчётов**. Из-за этого не удалось реализовать большинство запланированного на сентябрь, и весь график работ сдвинут на один месяц.

На **октябрь – ноябрь 2026 года** ключевая цель — **перевести модули «Основные средства» и «Учёт материальных ценностей» из стадии «метаданные + тесты» в рабочие пользовательские модули** с полноценным UI, проводками, периодом закрытия и покрытием тестами. Параллельно продолжается **строительство единого генератора отчётности** на базе нового конструктора (этапы 3.2), который заменит разрозненные пути формирования отчётов и активирует все 22 стандартных отчёта, включая временно

скрытые native-отчёты. Последующие этапы (зарплата, сбыт, сырьё, себестоимость) выполняются со сдвигом на месяц относительно августовского плана.